ADVANCED PROGRAMMING LANGUAGES

Spring 2026, CM 0632. MSc in Computer Science and Information Technology, Ca' Foscari University of Venice

Encapsulation and Abstract Data Types

We have discussed several techniques related to algebraic datatypes — their definition, manipulation, and the higher-order operations that make them useful. Distinctive of algebraic datatypes is their *open* structure: any client can pattern match on a value of an algebraic type and inspect its constituents directly. This is convenient in functional programming, but it has a cost: the internal representation of a type is exposed to all its clients.

In this lecture we look at the complementary discipline: **encapsulation**. The goal is to present a type to its clients only through the operations they are allowed to perform on it, hiding the internal representation entirely.

Before we dive into data abstraction and encapsulation, we enter a brief digression on a natural extension of the datatype definions discussed in the previous lectures.

A digressionc on Parametric datatypes

In our discussion on datatypes, we have so far focussed mostly on *monomorphic* types which fix the type of the values they contain, as in `IntLists`, and similarly for `IntTrees`. While such definitions are perfectly legal, (most, if not all) the operations on a such structures do not depend on what the elements *are*; they only depend on the structure of their container.

Parametric data types capture this generality. Just as a polymorphic function `length :: [a] -> Int` works for lists of any element type, a parametric data type can be declared to work for any element type by introducing a type variable in its definition. In Haskell:

```
data Tree a = Leaf | Node a (Tree a) (Tree a)
```

Type `Tree` now takes a *type parameter* `a`: `Tree Int` is a tree of integers, `Tree String` is a tree of strings, `Tree (Maybe Bool)` is a stack of optional booleans, and so on. The constructor `Tree` and the operations on Trees are written once and work for any choice of `a`.

Scala uses bracket notation for the same purpose:

```
enum Tree[A]:
  case Leaf
  case Node(value: A, left: Tree[A], right: Tree[A])
```

Elixir, being dynamically typed, has no parametric type declarations, but the underlying genericity is there at runtime, since any value can be put into any container.

Parametric data types and parametric polymorphism go hand in hand: the type variable in the data declaration matches the type variable in the operation signatures. We saw in the polymorphism lecture how parametricity constrains what such polymorphic operations can do. The same principles apply at the level of the data type: a `Tree a` is, by parametricity, oblivious to what `a` actually is.

However, we shoule not a subtle but important difference. Parametric type definitions are not types, nor do they introduce types. Instead, they introduce *type operators*, i.e. functions over types that construct new types by instantiating their parameter(s). Thus, in `Tree a` , `Tree` is the constructor that given the types `Int` , `Boolean` , . . . etc. constructs the corresponding set types `Tree Int` , `Tree Boolean` , . . . etc.

It is important to not confuse the two concepts of types and type constructors and to clarify the different uses of similar notation. To illustrate,
When we write

```
data List a = ...
```

we are *introducing* a new type constructor `List` mapping any type `a` to a list of `a` s. Instead, in a polymorphic function signature such as

```
lookup :: a -> List a -> a
```

the occurrence of `List a` denotes a type, namely the result of applying the constructor to the type variable `a` (implicitly quantified in the signature). Because Haskell uses juxtaposition for type application and binds parameters in distinct positions, the two roles of `List a` are visually separated.

In Scala, where `[A]` serves as both binder and application, the distinction is the same but easier to overlook, but the distinction is still there, namely:

```
trait List[A]...
```

introduces a type constructor, while in

```
def lookup[A](el : A, c : List[A]) : A = ...
```

the occurrence of `List[A]` denotes a type, that is the result of applying the constructor to the type variable `A` introduced by the method declaration.

Now back to data abstraction.

The concept: data abstraction

A **data abstraction** is a type defined together with a set of operations on it, such that clients can only interact with values of the type *through those operations*. The internal representation is invisible.

Two essential properties follow from this discipline:

- **Invariants are protected.** If clients cannot inspect or construct values directly, the implementer can guarantee that every value of the type satisfies certain properties (the *representation invariants*). For example, a `Set` implementation can guarantee that there are no duplicates if clients can only add elements through the exported `insert` operation.
- **Implementation freedom.** The implementer can change the internal representation — say, replacing a list with a balanced tree for efficiency — without affecting any client code. The clients depend only on the *interface*, not on the *representation*.

The interface (the set of operations) is the *contract*; the representation is an *implementation detail*. This is the foundational idea of abstract data types, and every modern language provides some mechanism to enforce it.

In this lecture we look at how the three languages of the course implement this discipline. We start with Haskell, where the mechanism is the simplest, then move to Elixir, and finally to Scala, where multiple mechanisms coexist.

Haskell: module export lists

In Haskell, encapsulation is realized through **module export lists**. A module declares which names it makes available to clients; everything else is private to the module.

Consider the canonical example: a stack of integers (or, polymorphically, a stack of any type).

```haskell
module Stack (Stack, empty, push, pop, top, isEmpty) where

newtype Stack a = MkStack [a]

empty :: Stack a
empty = MkStack []

push :: a -> Stack a -> Stack a
push x (MkStack xs) = MkStack (x : xs)

pop :: Stack a -> Stack a
pop (MkStack [])     = error "empty stack"
pop (MkStack (_:xs)) = MkStack xs

top :: Stack a -> a
top (MkStack [])    = error "empty stack"
top (MkStack (x:_)) = x

isEmpty :: Stack a -> Bool
isEmpty (MkStack xs) = null xs
```

The export list `(Stack, empty, push, pop, top, isEmpty)` is the key: it exports the *type name* `Stack`, but *not* the constructor `MkStack`. Clients see `Stack a` as an abstract type — they can declare values of type `Stack a`, pass them around, and call the exported functions on them, but they cannot:

- pattern match on `Stack` values (no access to the `MkStack` constructor)
- construct `Stack` values directly with `MkStack [1,2,3]`
- inspect the underlying list representation

The only way to interact with a `Stack` is through the five exported functions.

```haskell
import Stack

s = push 1 (push 2 empty)   -- OK: built through the interface
t = top s                    -- OK: 1
xs = case s of MkStack ys -> ys   -- ERROR: MkStack not in scope
```

This gives the implementer freedom: the internal representation is `[a]`, but it could just as well be `(a, Stack a)`, or a pair of lists for an efficient queue, or any other implementation of the same interface. Switching representations does not affect a single line of client code, because no client code mentions the representation.

Granularity. Haskell's mechanism is *module-level*: a name is either exported or it isn't. There are no `private` or `protected` modifiers on individual fields. This is coarser than the access modifiers of Java or Scala, but in practice it is sufficient — the right unit of encapsulation is usually the module, not the individual field.

A note on `newtype`. We used `newtype` rather than `data` to define `Stack`. `newtype` is a special form that introduces a new type with exactly one constructor and one field — no runtime overhead, no boxing. At runtime, a `Stack a` value is *identical* to the underlying `[a]`. The wrapping exists only in the type system, to enforce abstraction.

Elixir: opaque types and the module boundary

Elixir's encapsulation is similar in spirit to Haskell's, with the module as the unit of abstraction. The mechanism, however, is different in a subtle way.

In Elixir, every function defined in a module is by default callable from outside the module — there is no syntactic distinction between "exported" and "internal" functions, except through the `defp` (private function) keyword:

```
defmodule Stack do
  defstruct [:elements]

  def empty, do: %Stack{elements: []}
  def push(%Stack{elements: xs}, x), do: %Stack{elements: [x | xs]}
  def top(%Stack{elements: [x | _]}), do: x
  def top(%Stack{elements: []}), do: raise "empty stack"
  def pop(%Stack{elements: [_ | xs]}), do: %Stack{elements: xs}
  def pop(%Stack{elements: []}), do: raise "empty stack"
  def empty?(%Stack{elements: xs}), do: xs == []

  defp helper(...), do: ...   # private — not callable from outside
end
```

Notice that the argument to defstruct is a *keyword list*. We have already encountered the keyword syntax `key: value` in the context of maps and structs: `%{fst: 5, snd: 3}` is shorthand for `%{:fst => 5, :snd => 3}`, where the keys are the atoms `:fst` and `:snd`. The same shorthand applies inside lists: `[elements: []]` desugars to `[{:elements, []}]`, that is, a list whose elements are `{atom, value}` tuples. This idiom is pervasive in Elixir, where keyword lists serve as a lightweight way to pass named options to functions and macros, `defstruct` being one such macro. When a field has no default value, the bare-atom form is allowed as a special case: `[:elements]` is accepted in place of

`[{:elements, nil}]`, so that `defstruct [:elements]` and `defstruct elements: nil` define the same struct.

The public functions introduced by `def` define the interface; while the private functions defined by `defp` are internal helpers. So far, this looks like a finer-grained version of Haskell's export list.

The harder question is: *can clients still construct or inspect `%Stack{}` values directly?* The answer is yes: by default, structs are not opaque. A client can write `%Stack{elements: [1,2,3]}` and bypass the interface entirely, or pattern match `%Stack{elements: xs}` and access the internal list. The `defstruct` declaration only checks key validity; it does not hide the structure.

To enforce true abstraction, Elixir provides **opaque type specifications**:

```elixir
defmodule Stack do
  @opaque t :: %__MODULE__{elements: list()}

  defstruct [:elements]
  # ... functions as before
end
```

The `@opaque` declaration tells static analysis tools (Dialyzer, ElixirLS) that `Stack.t` should be treated as an abstract type outside the module, and clients should *not* depend on the representation. However, the enforcement is not hard: Dialyzer is an optional analysis, not part of compilation, so code that violates `@opaque` still compiles and runs.

This reflects Elixir's general philosophy: the type system is gradual and largely advisory (we discussed this in the lecture on data type design). Encapsulation is a discipline maintained by convention and tooling, not by the language. As Elixir's gradual type system matures, opaque types will likely gain stronger enforcement.

In practice, Elixir programmers rely on the module boundary for organization and on the `@opaque` annotation to signal intent — but the discipline of "use only the public interface" is largely up to the developers.

Scala (functional style): opaque types in objects

Scala provides a mechanism that is conceptually closest to Haskell's: **opaque type aliases**. An opaque type alias declares a type to be a synonym for another type *within a defining scope*, and an abstract type *outside*. Clients see an opaque type as completely distinct from its representation; only code within the defining scope sees the representation.

The natural way to use opaque types in functional style is to define them inside an `object` — Scala's counterpart of a module — together with the operations that act on them. Just as in Haskell or Elixir an opaque type is defined within a module that exports its operations but hides its representation, in Scala the object provides the same scoping discipline: the type and its operations live together, and the opaque modifier ensures that only code inside the object can see through the abstraction.

```
object Stacks:
  opaque type Stack[A] = List[A]

  object Stack:
    def apply[A](xs: A*): Stack[A] = xs.toList

  def empty[A]: Stack[A] = Nil

  def push[A](s: Stack[A], x: A): Stack[A] = x :: s

  def pop[A](s: Stack[A]): Stack[A] = s match
    case Nil    => sys.error("empty stack")
    case _ :: t => t

  def top[A](s: Stack[A]): A = s match
    case Nil    => sys.error("empty stack")
    case x :: _ => x

  def isEmpty[A](s: Stack[A]): Boolean = s.isEmpty
```

The `opaque type Stack[A] = List[A]` declaration says: inside `Stacks`, `Stack[A]` is an alias for `List[A]`; outside, it is an abstract type. Within the object, the operations can use list functions directly (`x :: s`, `s.isEmpty`) because they see the alias. Outside, a `Stack[Int]` is *not* a `List[Int]` — the type system rejects any attempt to confuse them:

```
import Stacks.*

val st = Stack(1, 2, 3)
val t  = top(s)                  // OK: 1
val u  = push(s, 0)              // OK
val xs : List[Int] = st          // ERROR: type mismatch
val v  = st.head                 // ERROR: head not a member of Stack[Int]
```

The `Stack` companion object provides `apply` so that clients can write `Stack(1, 2, 3)` to construct values.

Several features of this design are worth noting:

- **No runtime overhead.** Opaque types are erased: at runtime, a `Stack[A]` value *is* a `List[A]`. The abstraction exists only in the type system. This is the same property as Haskell's `newtype`.
- **No wrapping or unwrapping.** Inside the defining scope, the alias is transparent. Operations like `x :: s` work directly, with no need to wrap/unwrap a constructor.
- **Functional style.** The functions take the `Stack` as an explicit argument and return a new `Stack`, just as in Haskell. Clients write `push(s, 0)` rather than `s.push(0)`.

The structure parallels Haskell's module very closely: the `object Stacks` plays the role of the Haskell module, the `opaque type` plays the role of exporting `Stack` without `MkStack`, and the standalone functions are the exported operations.

Scala (with extension methods)

The functional style is clean, but many Scala programmers prefer the *method* call syntax — `s.push(0)` rather than `push(s, 0)`. Scala allows this through **extension methods**, which add methods to existing types without modifying their definition.

We can rewrite the `Stack` example to use extension methods:

```
object Stacks:
  opaque type Stack[A] = List[A]

  object Stack:
    def apply[A](xs: A*): Stack[A] = xs.toList
    def empty[A]: Stack[A] = Nil

  extension [A](s: Stack[A])
    def push(x: A): Stack[A] = x :: s

    def pop: Stack[A] = s match
      case Nil    => sys.error("empty stack")
      case _ :: t => t

    def top: A = s match
      case Nil    => sys.error("empty stack")
      case x :: _ => x

    def isEmpty: Boolean = s.isEmpty
```

Now clients use method syntax:

```
import Stacks.*

val s = Stack(1, 2, 3)
val t = s.top              // 1
val u = s.push(0)
val v = s.isEmpty          // false
```

The functional and extension-method versions are *functionally equivalent* — they expose the same interface and provide the same encapsulation guarantees. The only difference is the call syntax:

```
push(s, 0)        // functional style
s.push(0)         // extension method style
```

Which to prefer is a matter of taste and consistency with the surrounding codebase. The functional style stays close to Haskell; the extension-method style fits naturally with Scala's object-oriented heritage. Both are equally idiomatic Scala 3.

A second example: logarithms. A small numerical example illustrates the same pattern. We define an abstract type `Log` whose values are stored internally as logarithms — useful to avoid overflow when working with very large or very small numbers:

```
object Logarithms:

  opaque type Log = Double

  object Log:
    def apply(d: Double): Log = math.log(d)

  extension (x: Log)
    def value: Double = math.exp(x)
    def + (y: Log): Log = Log(math.exp(x) + math.exp(y))
    def * (y: Log): Log = x + y   // resolves to Double's + on the underlying values
```

The `apply` method converts a `Double` to a `Log` by taking its logarithm. Inside the object, `x` and `y` have type `Log` but are *known* to be `Double`s, so the arithmetic operations of `Double` are available. Outside, a `Log` is opaque — it cannot be confused with a `Double`, and the only operations available are those defined as extensions.

Note the cleverness of `*`: by the rule $\log(a \times b) = \log(a) + \log(b)$, multiplying two `Log` values is the same as *adding* their underlying representations. The line `def * (y: Log): Log = x + y` exploits this: inside the scope, `x + y` resolves to the `Double` addition (because members take precedence over extensions), giving us the correct logarithmic product without any wrapping.

Scala (with private classes)

Opaque types are not the only way to achieve encapsulation in Scala. The traditional object-oriented approach uses **access modifiers** on classes: a `private class` cannot be instantiated outside its enclosing scope, and `private` fields cannot be accessed from outside.

For the queue example, this looks as follows:

```
trait Queue[A]:
  def head: A
  def dequeue: Queue[A]
  def enqueue(x: A): Queue[A]

object Queue:

  def apply[A](as: A*): Queue[A] = new QueueImpl[A](as.toList, Nil)

  private class QueueImpl[A](
      val leading: List[A], val trailing: List[A]) extends Queue[A]:

    def reverse =
      if leading.isEmpty then new QueueImpl(trailing.reverse, Nil)
      else this

    def head: A = reverse.leading.head

    def dequeue: Queue[A] =
      val q = reverse
      new QueueImpl(q.leading.tail, q.trailing)

    def enqueue[A](a: A) =
      new QueueImpl[A](leading, a :: trailing)
```

The trait `Queue[A]` defines the public interface; the `private class QueueImpl` provides the implementation; and the companion object's `apply` method is the only way for clients to construct a `Queue`. Because `QueueImpl` is private to the `Queue` object, clients cannot instantiate it directly:

```
scala> val q = Queue(1, 2, 3)
val q: Queue[Int] = Queue$QueueImpl@...

scala> val qi = new QueueImpl(List(1), Nil)
-- Not Found Error: ----------------------------------
  |val qi = new QueueImpl(List(1), Nil)
  |             ^^^^^^^^^
  |             Not found: type QueueImpl
```

The interface (the trait) and the implementation (the private class) are clearly separated. Clients depend only on the trait; the implementation can be replaced without affecting them.

This approach is more verbose than opaque types — every operation requires constructing a new instance of `QueueImpl`, which incurs runtime overhead (object allocation, garbage collection). With opaque types, the same operations work directly on the underlying representation with no wrapping.

When to use which:

- **Opaque types** are preferred when the abstract type has a single, simple representation (a list, a pair, a `Double`). They are zero-cost.
- **Private classes (with traits)** are preferred when the abstract type benefits from polymorphism (subclasses can provide different implementations of the same interface) or when methods are conceptually "owned" by the data. They are more familiar to programmers from object-oriented backgrounds.

In practice, opaque types are the modern, idiomatic choice for simple abstract data types in Scala 3. Private classes remain useful when the abstraction calls for genuine subtyping or runtime polymorphism — topics we will return to in a later lecture.

Conclusion

The three languages provide three different mechanisms for encapsulation, but the underlying idea is the same: **a data abstraction is a type plus an interface, with the representation hidden behind the interface**.

	Mechanism	Granularity	Enforcement
Haskell	Module export list	Module-level	Compile-time, strict

	Mechanism	Granularity	Enforcement
Elixir	`defp` + `@opaque`	Function-level + type contract	Compile-time for `defp`; advisory for `@opaque`
Scala	Opaque types / private classes	Object-level	Compile-time, strict

In all three cases, encapsulation enables the two essential properties of data abstraction: invariants are protected from violation by clients, and the implementer is free to change the representation without breaking client code.

Combined with parametric types, encapsulation completes the toolkit for designing reusable, robust data abstractions: **parametric types** give us *generality* (the abstraction works for any element type), and **encapsulation** gives us *opacity* (the abstraction hides its representation).

REFERENCES

The Haskell module material draws on Chapter 11 of *Programming in Haskell (2nd Edition)*, G. Hutton, Cambridge University Press, 2016.

The Scala material on opaque types and access modifiers is from Chapter 18 of *Programming in Scala (5th Edition)*, M. Odersky, L. Spoon, B. Venners, F. Sommers, Artima Press, 2021, and from the Scala 3 reference.

A formal foundation for encapsulation in terms of *existential types* — beyond the scope of this lecture — can be found in L. Cardelli and P. Wegner's *On Understanding Types, Data Abstraction, and Polymorphism* (ACM Computing Surveys, 1985), and in chapters 24–25 of Pierce's *Types and Programming Languages*.